

What is Session? (org.hibernate.Session : i/f)

-----  
Session object is persistence manager for the hibernate application

Session object is the abstraction of hibernate engine for the Hibernate application

Session object provides methods to perform CRUD operations

Session just represents a thin wrapper around pooled out DB connection.

Session is associated implicitly with L1 cache (having same scope as the session lifetime) , referred as Persistence context.

Example of CRUD

save() - Inserting the record  
get() / load() - Retrieving the record  
update() - Updating the record  
delete() - Deleting the record

public void delete(Object ref) throws HibernateExc  
ref -- either persistent or detached pojo ref.

What is SessionFactory?(org.hibernate.SessionFactory : i/f)

-----  
It is a factory of session objects.(provider of session objects)

we use sessionFactory object to create session object(via openSession or getCurrentSession)  
singleton(1 instance per DB / application) ,immutable,inherently thrd safe.  
It is a heavy weight object, therefore it has to be created only once for an application & that too at the very beginning.

What is Configuration Object ?(org.hibernate.cfg.Configuration : class)

-----  
Configuration object is used to create the SessionFactory object.

Object Oriented Representation of Hibernate configuration file and mapping file is nothing but Configuration object.

When we call configure() method on configuration object ,hibernate configuration file(hibernate.cfg.xml placed in run time classpath) and mapping files are loaded in the memory.

API of Configuration class

1. Create Configuration class instance using default constructor.
2. To populate the config with the properties added in xml file , use -  
public Configuration configure() throws HibernateException
3. To build the singleton SessionFactory(SF) from config

public SessionFactory buildSessionFactory() throws HibernateException

## POJO/Entity Life cycle

-----

### 1.Transient State

-----

An object is said to be in transient state if it is not associated with the session, and has no matching record in the database table.

For example

-----

```
Account account=new Account();
```

```
account.setAccno(101);  
account.setName("Amol");  
account.setBalance(12000);
```

### 2.Persistent State

-----

An object is said to be in persistent state if it is associated with session object (L1 cache) and will result into a matching record in the database table.(i.e upon commit)

```
session.save(account);tx.commit();
```

or

```
Account account=session.get(Account.class,102);  
OR via HQL/JPQL
```

Note

-----

When the POJO is in persistent state it will be in synchronization with the matching record in DB i.e if we make any changes to the state of persistent POJO it will be reflected in the database.(after committing tx) -- i.e automatic dirty checking will be performed(resulting in insert/update/delete)

### 3.Detached state

-----

Object is not associated with session but has matching record in the database table.

If we make any changes to the state of detached object it will NOT be reflected in the database.

Triggers for transition from persistent --> detached ?

1. Upon session closing
2. evict
3. clear

```
session.clear();  
session.evict(Object);  
session.close();
```

Note :

-----

By calling update method on session object it will go from detached state to persistent state.

By calling delete method on session object it will go from persistent state to transient state.

Explain the following methods of Session API

`public void persist(Object ref)` -- Persists specified transient POJO on underlying DB , upon committing the transaction.

-----

`void clear()`

-----

When `clear()` is called on session object all the objects associated with the session object become detached.

But Database Connection is not closed.

(Completely clears the session. Evicts all loaded instances and cancel all pending saves, updates and deletions)

`void close()`

-----

When `close()` is called on session object all the objects associated with the session object become detached and also closes the Database Connection.

`public void evict(Object ref)`

-----

It detaches a particular persistent object detached or disassociates from the session.

(Remove this instance from the session cache. Changes to the instance will not be synchronized with the

database. )

void flush()

-----

When the object is in persistent state ,  
whatever changes we made to the object  
state will be reflected in the database only  
at the end of transaction.

If we want to reflect the changes before the end of transaction  
(i.e before committing the transaction )  
call the flush method.

(Flushing is the process of synchronizing the underlying DB state with persistable state of session cache  
)

boolean contains(Object ref)

-----

The method indicates whether the object is  
associated with session or not.

void refresh(Object ref) -- ref --persistent or detached

-----

This method is used to get the latest data from database and make  
corresponding modifications to the persistent object state.  
(Re-read the state of the given instance from the underlying database)

-----

public void update(Object ref)

Note :-

If object is in persistent state no  
need of calling the update method .  
As the object is in sync with the  
database whatever changes made to the object  
will be reflect to database at the  
end of transaction.

eg --- updateAccount(Account a,double amt)

```
{  
    sess, tx  
    sop(a);set amt  
    sess.update(a);  
    sop(a);  
}
```

When the object is in detached state record  
is present in the table  
but object is not in sync with database,  
therefore update() method can be called  
to update the record in the table

Which exceptions update method can raise?

1. `StaleStateException` -- If u are trying to update a record (using `session.update(ref)`), whose id doesn't exist.

i.e update can't transition from transient ---> persistent

It can only transition from detached ---> persistent.

eg -- `update_book.jsp` -- supply updated details + id which doesn't exists on db.

2. `NonUniqueObjectException` -- If there is already persistence instance with same id in session.

eg -- `UpdateContactAddress.java`

-----

`public Object merge(Object ref)`

Can Transition from transient --> persistent & detached ---> persistent.

Regarding Hibernate merge

1. The state of a transient or detached instance may also be made persistent as a new persistent instance by calling `merge()`.

2. API of Session

`Object merge(Object object)`

3.

Copies the state of the given object (can be passed as transient or detached) onto the persistent object with the same identifier.

3. If there is no persistent instance currently associated with the session, it will be loaded.

4. Return the persistent instance. If the given instance is unsaved, save a copy of and return it as a newly persistent instance. The given instance does not become associated with the session.

5. will not throw `NonUniqueObjectException` -- Even If there is already persistence instance with same id in session.

-----

`public void saveOrUpdate(Object ref)`

-----

The method persists the object (insert) if matching record is not found (& id init'd to default value) or fires update query

If u supply Object, with non-existing ID -- Fires `StaleStateException`.

`lock()`

-----

when `lock()` method is called on the session object for a persistent object,

until the transaction is committed in

the hibernate application, externally the matching record in the table cannot be modified.

`session.lock(object, LockMode);`

eg - `session.lock(account, LockMode.UPGRADE);`